Μέρος Α

Ξεκινήσαμε κατασκευάζοντας την κλάση StringDoubleEndedQueueImpl, που είναι η υπολοποίηση της διεπαφής.

Φτιάχνουμε τρία πεδία της κλάσης, τα “first”, “last”, και “size”. Τα δύο πρώτα δείχνουν την αρχή και το τέλος της ουράς αντίστοιχα, ενώ το τελευταίο το μέγεθος της ουράς (ακέραιος αριθμός).

Στην συνέχεια κατασκευάζουμε την εμφωλευμένη κλάση Node, που χρησιμοποιείται για να δείξει τους κόμβους της διπλά συνδεδεμένης λίστας. Η κλάση περιέχει ένα αντικείμενο τύπου String, μία αναφορά στον επόμενο κόμβο («next»), και μια στον προηγούμενο («prev»). Με αυτόν τον τρόπο ενώνουμε τους κόμβους, και προκύπτει η δομή της λίστας.

Αρχικοποιοιούμε τα τρία πεδία κλάσης μέσω του κατασκευαστή, και τα ορίζουμε αρχικά ως NULL, NULL και 0 αντίστοιχα.

Συνεχίζουμε φτιάχνοντας την μέθοδο isEmpty, που μας επιστρέφει την τιμή TRUE αν η ουρά είναι άδεια, και την τιμή FALSE αν δεν είναι.

Η μέθοδος add.First κατασκευάζει έναν καινούριο κόμβο, τον οποίο προσθέτει στην αρχή της ουράς, και της δίνουμε μια τιμή (item) για να βάλει μέσα στον νέο κόμβο. Επίσης αλλάζει τις αναφορές προς τον επόμενο και τον προηγούμενο κόμβο. Εάν η ουρά είναι αρχικά άδεια, ο κόμβος που εισάγουμε είναι ο μόνος κόμβος της ουράς, άρα ορίζουμε τόσο την αρχή (first) όσο και το τέλος της ουράς (last) ως τον κόμβο που εισαγάγαμε. Εάν η ουρά δεν είναι άδεια, ορίζουμε τον νέο κόμβο ως τον πρώτο της ουράς (first), και ταυτόχρονα ως τον προηγούμενο του κόμβου που ήταν πρώτος πριν εισάγουμε τον κόμβο (first.prev). Η μέθοδος ολοκληρώνεται σε χρόνο O(1), καθώς έχουμε άμεση πρόσβαση στον πρώτο κόμβο της ουράς, και μπορούμε απλά να προσθέσουμε τον καινούριο πρώτο κόμβο αλλάζοντας τους pointers του προηγούμενου πρώτου κόμβου. Δεν χρειάζεται, δηλαδή, κάποιου είδους επανάληψη για όλους τους κόμβους της ουράς: είτε έχουμε 100 κόμβους στην ουρά είτε 2, η προσθήκη νέου κόμβου στην αρχή της ουράς αλλάζει τους pointers μόνο του πρώτου κόμβου, και γίνεται στον ίδιο χρόνο.

Ομοίως, η μέθοδος add.Last προσθέτει έναν νέο κόμβο στο τέλος της ουράς. Εάν η ουρά είναι αρχικά άδεια, ο κόμβος που εισάγουμε είναι ο μόνος κόμβος της ουράς, άρα ορίζουμε τόσο την αρχή (first) όσο και το τέλος της ουράς (last) ως τον κόμβο που εισαγάγαμε. Εάν η ουρά δεν είναι άδεια, ορίζουμε τον νέο κόμβο ως τον τελευταίο της ουράς (last), και ταυτόχρονα ως τον επόμενο του κόμβου που ήταν τελευταίος πριν εισάγουμε τον κόμβο (last.next). Η μέθοδος ολοκληρώνεται σε χρόνο O(1), καθώς έχουμε άμεση πρόσβαση στον τελευταίο κόμβο της ουράς, και μπορούμε απλά να προσθέσουμε τον καινούριο τελευταίο κόμβο αλλάζοντας τους pointers του προηγούμενου τελευταίου κόμβου. Δεν χρειάζεται, δηλαδή, κάποιου είδους επανάληψη για όλους τους κόμβους της ουράς, δηλαδή ο χρόνος δεν εξαρτάται από το μέγεθος της ουράς.

Η μέθοδος remove.First αφαιρεί τον πρώτο κόμβο της ουράς. Εάν η ουρά είναι αρχικά άδεια, παίρνουμε το error message NoSuchElementException, και δεν μπορούμε να αφαιρέσουμε τον πρώτο κόμβο, αφού δεν υπάρχει. Η διαγραφή γίνεται αλλάζοντας τον pointer “first”, ώστε να δείχνει στον δεύτερο κόμβο της λίστας (first.next). Εάν μετά από αυτήν την διαδικασία ο πρώτος κόμβος είναι άδειος, η ουρά είναι άδεια, άρα ορίζουμε και τον πρώτο και τον τελευταίο κόμβο της ουράς ως NULL. Εάν η λίστα δεν είναι άδεια, ο “prev” pointer του καινούριου πρώτου κόμβου (δηλαδή του αρχικά δεύτερου) ορίζεται σε NULL.

Η μέθοδος remove.Last λειτουργεί με παρόμοιο τρόπο, αλλάζοντας τους pointers του τελευταίου και προτελευταίου κόμβου. Και οι δύο μέθοδοι ολοκληρώνονται σε χρόνο O(1), καθώς έχουμε άμεση πρόσβαση στον πρώτο και στον τελευταίο κόμβο της ουράς, αντίστοιχα. Η λογική είναι ίδια με εκείνη των μεθόδων εισαγωγής. Οι μέθοδοι getFirst και getLast, που επιστρέφουν το πρώτο και το τελευταίο στοιχείο της ουράς, ολοκληρώνονται σε χρόνο O(1), διότι και αυτές ασχολούνται μόνο με τον πρώτο και τον τελευταίο κόμβο για να πραγματοποιηθούν. Η μέθοδος size είναι επίσης O(1), καθώς το πεδίο της κλάσης size (αρχική τιμή: NULL) ενημερώνεται σε κάθε πρόσθεση/αφαίρεση στοιχείου, άρα ο χρόνος είναι σταθερός σε κάθε ενημέρωση.

Τέλος, τυπώνουμε τα στοιχεία της ουράς (τα στοιχεία μέσα στους κόμβους), και επιστρέφουμε το μέγεθος της ουράς. Επίσης, αλλάζουμε την υλοποίηση της διεπαφής, χρησιμοποιώντας generics. Τροποποιήσαμε τις κατάλληλες παραμέτρους, ώστε η ουρά μας να δέχεται οποιονδήποτε τύπο δεδομένων, και όχι μόνο String.

Μέρος Β

Ως γνωστό, η υλοποίηση του μέρους Α είναι η υλοποίηση μιας διπλά συνδεδεμένης ουράς. Ως διπλά συνδεδεμένη ουρά έχει 2 άκρα. Στην δική μας υλοποίηση τα δύο αυτά άκρα ονομάζονται First και Last, αναπαριστώντας το πρώτο και τελευταίο στοιχείο της ουράς αντίστοιχα. Έχοντας ως ιδέα τα παραδείγματα στις διαφάνειες των διαλέξεων, για την μετατροπή μιας Infix αριθμητικής έκφρασης στην αντίστοιχη Postfix και τον υπολογισμό μιας Postfix αριθμητικής παράστασης, καθώς και προσωπικό μας ψάξιμο, αποφασίσασμε στο Μέρος Β να κάνουμε χρήση στοίβας για την επίλυση του προβλήματος, χρησιμοποιώντας την διπλά συνδεδεμένη λίστα του μέρους Α ως στοίβα, κάνοντας push και pop στοιχεία μόνο από την μία μεριά της λίστας, θεωρώντας την συγκεκριμένη μεριά ως την κορυφή της στοίβας(αφού βασική αρχή της στοίβας είναι πως κάνουμε push και pop αποκλειστικά από την κορυφή της στοίβας). Καθώς όπως αναφέραμε σε μία στοίβα κάνουμε push και pop από την κορυφή της στοίβας, όπου ως κορυφή κάθε φορά θεωρείται το πρώτο στοιχείο της στοίβας, καταλήξαμε στην απόφαση να θεωρήσουμε ως κορυφή της στοίβας το άκρο First της ουράς, θεωρώντας το πιο κοντινό σημασιολογικά, επομένως βάζουμε και βγάζουμε στοιχεία μόνο από το αριστερό άκρο της ουράς.

Στη συνέχεια μετά τον έλεγχο για την έγκυρη είσοδο του χρήστη, προτού χρησιμοποιήσουμε την υλοποίηση του μέρους Α για την μετατροπή σε ενθεματική μορφή, αντιστρέψαμε την προθεματική έκφραση μετατρέποντάς την σε μεταθεματική, ώστε μέσω αυτής να κάνουμε χρήση του μέρους Α για την εύρεση της ενθεματικής παράστασης ακολουθώντας την εξής διαδικασία, έχοντας ως παράδειγμα την προθεματική παράσταση *+12/34. Εφόσον η συγκεκριμένη παράσταση θα αντιστραφεί, προκύπτει η παράσταση 43/21+* η οποία είναι σε μεταθεματική μορφή, όπου μέσω αυτής θα ακολουθήσουμε τη διαδικασία για την εύρεση της ενθεματικής παράστασης.

Ακολουθεί η διαδικασία για να κάνουμε την μετατροπή σε ενθεματική μορφή, χρησιμοποιώντας την υλοποίηση από το μέρος Α.

Prefix to Infix

Prefix: *+12/34. Μετατροπή σε Postfix -> Postfix: 43/21+*.

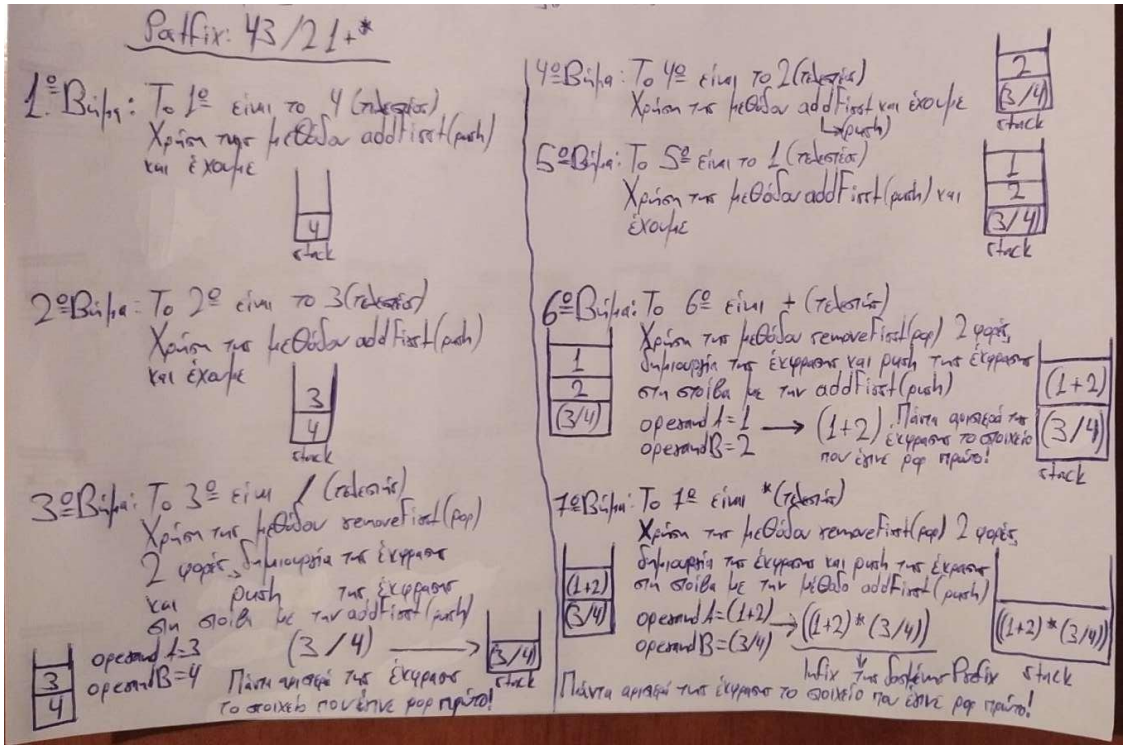
Σε αυτό το σημείο ξεκινάει η χρήση της υλοποίησης από το μέρος Α (Έχοντας πρώτα μετατρέψει, όπως αναφέρθηκε και προηγουμένως, την Postfix παράσταση που προέκυψε σε ένα πίνακα χαρακτήρων).

Η διάσχιση του πίνακα χαρακτήρων γίνεται μέσω της χρήσης του βρόχου for.

Όταν το στοιχείο του πίνακα είναι τελεστέος, τον κάνουμε push στην “στοίβα” με την μέθοδο addFirst(). Όταν το στοιχείο του πίνακα είναι τελεστής, κάνουμε “pop” 2 στοιχεία από τη “στοίβα” με την μέθοδο removeFirst() και δημιουργούμε την έκφραση που προκύπτει έχοντας τον τελεστή που μόλις συναντήσαμε ανάμεσα από τα δύο αυτά στοιχεία (τελεστέους). Έπειτα κάνουμε “push” στη “στοίβα” τη συγκεκριμένη έκφραση με χρήση της μεθόδου addFirst(). Η διαδικασία αυτή

ακολουθείται μέχρι να ολοκληρωθεί η διάσχιση του πίνακα χαρακτήρων ακολουθώντας τη διαδικασία που περιγράφτηκε προηγουμένως, και καταλήξουμε με ένα και μοναδικό στοιχείο (που είναι αριθμητική παράσταση) στη στοίβα, το οποίο θα είναι και η Infix μορφή της Prefix παράστασης που έδωσε ως είσοδο ο χρήστης.

Ακολουθεί σχηματικά η διαδικασία που περιγράψαμε, από το σημείο που ξεκινάει η διάσχιση του πίνακα χαρακτήρων, με χρήση του βρόχου for, για τον έλεγχο των στοιχείων του πίνακα ένα προς ένα και την εκτέλεση της διαδικασίας που πρέπει να εκτελεστεί στην περίπτωση που είναι τελεστής και στην περίπτωση που είναι τελεστέος.



Μέρος Γ

Για να ελέγξουμε εάν μια DNA ακολουθία είναι Watson-Crick παλίνδρομη ή όχι μέσω της ουράς, πρέπει να εισάγουμε τα στοιχεία στην ουρά, και να κάνουμε τα παρακάτω:

→ Να αντικαταστήσουμε κάθε νουκλεοτίδιο με το συμπλήρωμά του και να
→ Αντιστρέψουμε την σειρά.

Εάν προκύψει η αρχική ακολουθία, έχουμε Watson-Crick παλίνδρομη ακολουθία.

Αρχικά, δημιουργούμε μια κενή λίστα με το όνομα queue.

Η πρώτη μας σκέψη ήταν να βρούμε για κάθε στοιχείο της ακολουθίας το συμπληρωματικό του, και να εισάγουμε την ακολουθία με τα συμπληρωματικά στοιχεία στην λίστα με την μέθοδο add.First, δηλαδή από μπροστά. Ύστερα να τα αφαιρέσουμε ένα-ένα με την μέθοδο remove.First για να προκύψει η αντεστραμμένη ακολουθία, ώστε να ελέγξουμε αν αυτή ταυτίζεται με την αρχική.

Ωστόσο, προτιμήσαμε τελικά να χρησιμοποιήσουμε και τα δύο άκρα της ουράς, ώστε να την εκμεταλλευτούμε πλήρως.

Αρχικά, ορίσαμε τις τιμές που μπορούν να πάρουν τα νουκλεοτίδια (A, T, C, G), και φτιάξαμε την μέθοδο isComplementary για να ελέγξουμε αν κάθε ζεύγος τους είναι συμπληρωματικό (output TRUE) ή όχι (output FALSE).

Φτιάξαμε έναν βρόχο for για να ελέγξουμε αν το κάθε στοιχείο της ακολουθίας είναι τιμή που μπορεί να πάρει ένα νουκλεοτίδιο ή όχι. Σε κάθε επανάληψη του βρόχου, παίρνουμε με την σειρά ένα-ένα τα στοιχεία της ακολουθίας, μέχρι να φτάσουμε στο τελευταίο. Εάν η τιμή του εκάστοτε στοιχείου της ακολουθίας είναι δεκτή, την προσθέτουμε στην ουρά μέσω της μεθόδου add.Last, για να μην χαλάσει η σειρά της αρχικής ακολουθίας. Διαφορετικά, επιστρέφουμε το μήνυμα λάθους «Invalid DNA sequence: " + την μη δεκτή τιμή».

Επειδή ξέρουμε ότι το κενό string είναι Watson-Crick παλίνδρομο, ελέγχουμε με την συνθήκη if αν έχουμε κενό string και αν ισχύει αυτό, επιστρέφουμε απευθείας ότι έχουμε W-C παλίνδρομο.

Επίσης, χρησιμοποιούμε μια καινούρια συνθήκη if για να ελέγξουμε αν η ακολουθία αποτελείται από μονό αριθμό στοιχείων. Σε περίπτωση που συμβαίνει αυτό, επιστρέφουμε απευθείας ότι δεν είναι W-C παλίνδρομη.

Εφόσον έχουμε γεμίσει την ουρά με στοιχεία και δεν έχουμε κενό string ή μονό αριθμό στοιχείων, φτιάχνουμε έναν βρόχο while που παίρνει ως παράμετρο το μέγεθος της ουράς. Όσο έχουμε ουρά με αριθμό στοιχείων μεγαλύτερο ή ίσο του 2 (που δεν είναι μονός), αφαιρούμε ένα στοιχείο από την αρχή της ουράς και ένα από το τέλος, και ελέγχουμε αν είναι συμπληρωματικά. Αυτή η διαδικασία ελέγχει απευθείας την συμπληρωματικότητα μετά την αντιστροφή, καθώς κατά την αντιστροφή το πρώτο στοιχείο θα πάει στην θέση του πρώτου, ενώ το πρώτο στου τελευταίου, θα αυτό θα ισχύει για όλες τις επαναλήψεις του while μέχρι να αδειάσει η ουρά. Επομένως, το να ελέγξουμε την συμπληρωματικότητα του πρώτου και του τελευταίου στοιχείου σε κάθε επανάληψη εξετάζει και την συμπληρωματικότητα και την αντιστροφή. Αν η τελική ακολουθία είναι ίδια με την αρχική λαμβάνουμε true (και έχουμε W-C παλίνδρομη ακολουθία) , διαφορετικά false (και δεν έχουμε W-C παλίνδρομη ακολουθία).

Βάζουμε την παραπάνω διαδικασία σε μια Boolean μέθοδο, την isWatsonCrickPalindrome, που επιστρέφει την τιμή TRUE αν έχουμε W-C παλίνδρομη ακολουθία, και την τιμή FALSE εάν δεν έχουμε.

Τέλος, βάζουμε όλα τα παραπάνω στην κλάση DNAPalindrome, και προσθέτουμε στον κώδικα την βοηθητική κλάση Scanner για το input του χρήστη. Εάν το input του χρήστη είναι W-C παλίνδρομη ακολουθία, τυπώνεται το μήνυμα "Yes, it is a Watson-Crick complemented palindrome.", ενώ αν δεν είναι τυπώνεται το μήνυμα "No, it is not a Watson-Crick complemented palindrome.".

Παράδειγμα: Έστω η ακολουθία ATCGAT.

Αρχικά, ο βρόχος for ελέγχει πρώτα το στοιχείο A, και το ορίζει ως currentNucleotide. Η μέθοδος isValidNucleotide μας λέει ότι πρόκειται για δεκτή τιμή νουκλεοτιδίου, επομένως προστίθεται στην κενή ουρά από το τέλος, και αποτελεί προς το παρόν το μοναδικό στοιχείο της. Ύστερα, ο βρόχος ελέγχει το στοιχείο T,

και το ορίζει ως currentNucleotide. Με την ίδια λογική, εφόσον ότι πρόκειται για δεκτή τιμή νουκλεοτιδίου, το στοιχείο προστίθεται στο τέλος της λίστας, μετά το A (τώρα η λίστα έχει 2 στοιχεία). Ο βρόχος επαναλαμβάνεται για κάθε στοιχείο της ακολουθίας, μέχρι να φτάσουμε στο τελευταίο. Εφόσον είναι δεκτά όλα τα στοιχεία, μπαίνουν όλα στην λίστα, και δεν παίρνουμε κάποιο μήνυμα λάθους.

Η σειρά των στοιχείων διατηρείται μέσα στην ουρά.

Δεν έχουμε κενό string ή μονό αριθμό στοιχείων, άρα η διαδικασία προχωράει στο while loop.

Πρώτη επανάληψη while:

Αφαιρούμε από την ουρά το πρώτο και το τελευταίο στοιχείο (A και T). Αυτά είναι συμπληρωματικά, επομένως η διαδικασία συνεχίζεται.

Δεύτερη επανάληψη while:

Αφαιρούμε από την ουρά το πρώτο και το τελευταίο στοιχείο (T και A). Αυτά είναι συμπληρωματικά, επομένως η διαδικασία συνεχίζεται.

Τρίτη επανάληψη while:

Αφαιρούμε από την ουρά το πρώτο και το τελευταίο στοιχείο (C και G). Αυτά είναι συμπληρωματικά, επομένως η διαδικασία συνεχίζεται.

Η ουρά δεν έχει κανένα στοιχείο πλέον μέσα, άρα σταματάει ο βρόχος.

Το loop επιστρέφει true, άρα τυπώνεται το μήνυμα "Yes, it is a Watson-Crick complemented palindrome.".

Άννα-Λευκοθέα Παπαβασιλείου 6190101

Αναστάσιος-Ραφαήλ Παπαδόπουλος 3160130